

Imbalanced Data in Machine Learning

1. What is Imbalanced Data?

Imbalanced data occurs in a **classification problem** where one class has significantly more samples than another.

Example:

- IIT placement dataset: 500 students total
- 450 students **got placed** (Majority Class)
- 50 students **did NOT get placed** (Minority Class)

This unequal distribution = **Imbalanced Data**

Works for both **Binary** and **Multi-class** classification problems.

2. Two Major Problems with Imbalanced Data

● Problem 1 — Bias Toward Majority Class

- ML algorithm sees more data from the majority class
- It learns more about majority → **ignores minority**
- Result: Model performs **well on majority, poorly on minority**
- This is called **bias toward the majority class**

● Problem 2 — Accuracy is NOT a Reliable Metric

Example:

- 900 students placed, 100 not placed
- A "dumb" model that always predicts "placed" for everyone
- Accuracy = **90%** (looks great!)
- BUT it **never correctly identifies** students who won't get placed

Key takeaway: Accuracy is misleading with imbalanced data. Use **Precision, Recall, ROC-AUC** instead.

Real proof:

- Logistic Regression on imbalanced data → **96% accuracy**
- BUT Recall for minority class = only **29%**
- Meaning: out of 100 minority samples, only 29 were correctly classified

3. Why Should You Learn Imbalanced Data?**Reason 1 — It's Everywhere in Industry**

Domain	Problem	Why Imbalanced?
Finance	Fraud Detection	Normal transactions >> Fraudulent
Banking	Credit Risk Assessment	Most people repay loans, few default
Healthcare	Rare Disease Prediction	Most people don't have rare diseases
Manufacturing	Predictive Maintenance	Most machines work fine, few break
Consumer Internet	Churn Prediction	Most customers stay, few leave
Earth Sciences	Earthquake Prediction	Earthquakes are rare events

In all these cases — **predicting the minority class is more important!**

Reason 2 — Very Important for Interviews

- Interviewers frequently ask about imbalanced data
- It shows real-world ML knowledge beyond just algorithms

4. Five Techniques to Handle Imbalanced Data**◆ Technique 1 — Under Sampling**

Idea: Reduce the majority class to match the minority class size

How it works:

- Majority class: 900 samples
- Minority class: 100 samples

- Randomly remove samples from majority until both = 100
- Train model on this new balanced dataset

Advantages	Disadvantages
Reduces bias toward majority	Loses a lot of data
Faster training (smaller dataset)	Sampling may introduce its own bias

◆ Technique 2 — Over Sampling (Random Over Sampling)

Idea: Increase the minority class by duplicating its samples

How it works:

- Minority class: 100 samples
- Majority class: 900 samples
- Randomly duplicate minority samples until both = 900
- Train model on this balanced dataset

Advantages	Disadvantages
Reduces bias	Dataset size almost doubles
No data loss	Duplication can cause overfitting (especially in Decision Trees)

◆ Technique 3 — SMOTE

Full Form: Synthetic Minority Over-sampling TEchnique

Key difference from Random Over Sampling:

- Random Over Sampling → **duplicates** existing minority points
- SMOTE → **generates NEW synthetic data points** using interpolation

How SMOTE works (Step by Step):

Step 1: Train a **KNN (k=5)** on minority class samples only → find 5 nearest neighbors for each minority point

Step 2: Synthetic Data Generation:

- Randomly pick a minority sample (e.g., point A)
- Randomly pick one of its 5 neighbors (e.g., point B)
- Generate a new point between A and B using:

$$\text{New Point} = A + \lambda \times (A - B)$$

where λ is a random number between 0 and 1

Example:

- $A = (1, 1)$, $B = (2, 2)$, $\lambda = 0.5$
- $\text{New Point} = (1, 1) + 0.5 \times (1-2, 1-2) = (1.5, 1.5)$

Step 3: Repeat until majority and minority classes are balanced

Advantages	Disadvantages
Reduces bias	Cannot handle categorical data
No duplication → less overfitting	KNN is slow on high-dimensional data
Generates diverse new data	Performance depends on value of k
	Sensitive to outliers
	No guarantee new points follow true distribution

◆ Technique 4 — Ensemble Methods (Balanced Random Forest)

Idea: Modify Random Forest so that each bootstrap sample is **balanced**

How Balanced Random Forest works:

Normal Random Forest: samples data randomly (imbalanced samples likely)

Balanced Random Forest:

- Data: 900 majority + 300 minority = 1200 total
- **Sample 1:** Take all 300 minority + randomly sample 300 from majority → train Tree 1
- **Sample 2:** Take all 300 minority + randomly sample 300 from majority → train Tree 2
- **Sample 3:** Same → train Tree 3
- Final prediction: **majority voting** across all trees

Key benefit: No separate resampling step needed — balancing happens **inside** the algorithm itself

◆ Technique 5 — Cost Sensitive Learning

Idea: Modify the learning process itself so the algorithm pays more attention to the minority class

Method A — Class Weights

- Assign **higher weight** to minority class
- Assign **lower weight** to majority class
- Algorithm penalizes mistakes on minority class more heavily

Example:

- Majority class (class 1): weight = 1
- Minority class (class 0): weight = 10

Available in: **Logistic Regression, SVM, Decision Tree, Gradient Boosting** and most Scikit-learn models

Higher weight → bigger gradient update → model learns minority class better

Method B — Custom Loss Function

- Modify the loss function directly to penalize certain errors more
- Available in: **XGBoost, LightGBM, Gradient Boosting**

Example: If False Positive is more critical (like spam filtering):

Advantages	Notes
Very easy to implement (class_weight parameter)	Class weights available in most sklearn models
No data modification needed	Custom loss → more technical but more flexible
Fine-grained control	KNN and Naive Bayes do NOT support class weights

5. Summary of All 5 Techniques

Technique	Core Idea	Key Tool
Under Sampling	Reduce majority class	RandomUnderSampler
Over Sampling	Duplicate minority class	RandomOverSampler
SMOTE	Generate new synthetic minority points	SMOTE
Balanced Random Forest	Balance inside ensemble	BalancedRandomForestClassifier
Cost Sensitive Learning	Weight minority class higher in loss	class_weight / custom loss

6. Key Metrics to Use (NOT Accuracy)

When working with imbalanced data always evaluate using:

-  **Recall** — How many actual minority cases did we catch?
-  **Precision** — Of predicted minority, how many were correct?
-  **F1 Score** — Balance between Precision and Recall
-  **ROC-AUC Score** — Overall model discrimination ability
-  **Accuracy** — Misleading! Avoid as primary metric

7. Recommended Library

imbalanced-learn (imblearn)

- Contains implementations of ALL techniques
- Seamlessly integrates with Scikit-learn
- Documentation: imbalanced-learn.org